

HIP Lab 1–6 填空与思考题答案

Lab 1: Vector Addition

Thread Index Calculation

blockIdx.x	blockDim.x	threadIdx.x	Global Index
0	256	100	100
2	128	50	306
5	64	0	320

Exercise 1: GPU Specifications

Property	Your GPU Value
Device Name	AMD Radeon 890M Graphics
Compute Units	16 physical CUs (HIP reports 8 WGP's)
Wavefront Size	32
Max Threads per Block	1024
Max Shared Memory per Block	65536 bytes (64 KiB)

题目： 当所有 CU 都被充分利用时，GPU 最多可同时驻留多少个线程？

回答：

$$8 \times 64 \times 32 = 16384 \text{ threads.}$$

Exercise 2: Implement the Kernel

```
__global__ void vector_add(const float* A, const float* B,
                          float* C, int N) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx < N) {
        C[idx] = A[idx] + B[idx];
    }
}
```

Exercise 3: Launch Configuration

Block Size	Grid Size Formula	Grid Size	Total Threads	Idle Threads
64	$\lceil 1000/64 \rceil$	16	1024	24
128	$\lceil 1000/128 \rceil$	8	1024	24
256	$\lceil 1000/256 \rceil$	4	1024	24
512	$\lceil 1000/512 \rceil$	2	1024	24

题目： 哪种 block size 的线程利用率最高？为什么它不一定总是最佳选择？

回答： 四种配置的线程利用率相同，均为 97.66%。线程利用率相同不代表性能相同，实际性能还会受到 occupancy、寄存器、LDS 和调度开销等因素影响。

Exercise 4: Analyze Your Results

题目 1： 哪种 block size 的平均执行时间最低？

回答： Block size 1024，平均执行时间为 0.1411 ms。

题目 2： 哪种 block size 获得了最高内存带宽？

回答： Block size 1024，最高带宽约为 89.18 GB/s。

题目 3： 不同 block size 之间的差异是否具有明显统计显著性？

回答： 没有明显统计显著性，因为各组误差棒大量重叠。

题目 4： 为什么 Vector Add 的性能更多取决于带宽而不是 block size？

回答： 每个元素只执行一次加法，却需要读取两个 float 并写入一个 float，算术强度很低，因此性能主要受内存带宽限制。

Exercise 5: Wavefront Analysis

题目 1： Block size 为 100，在 wave32 下需要多少个 wavefront？浪费多少个 lane？

回答： 4 个 wavefront，浪费 28 个 lane。

题目 2： 若使用 `-mwavefrontsize64`，答案如何变化？

回答： 2 个 wavefront，浪费 28 个 lane。

题目 3： 同一 wavefront 内的线程进入不同分支时会发生什么？

回答： 会发生分支发散。GPU 使用执行掩码分别执行各条分支路径，使部分 lane 暂时空闲，从而降低执行效率。

Exercise 6: Occupancy Calculation

Block Size	Wavefronts/Block	Blocks that fit	Active Wavefronts	Notes
32	1	64	64	受 max blocks 限制
64	2	32	64	理论 occupancy 100%
128	4	16	64	理论 occupancy 100%
256	8	8	64	理论 occupancy 100%
512	16	4	64	理论 occupancy 100%
1024	32	2	64	理论 occupancy 100%

Exercise 7: Analyze Sub-Wavefront Results

Block Size	Mean Time (ms)
8	0.1599
16	0.1443
32	0.1452
256	0.1458

题目 1： Block size 8、16 相比 wavefront 对齐的配置是否存在明显性能损失？

回答： Block size 8 存在明显性能损失，约慢 11%；block size 16 与对齐配置的误差棒重叠，本次实验未观察到明显性能损失。

题目 2： 如果使用 wavefront size 为 64 的 MI100，哪些 block size 会成为 sub-wavefront？

回答： 8、16 和 32。

Challenge Exercise

填空：

```
int stride = gridDim.x * blockDim.x;
```

题目： 这种方法相对于线程与元素 1:1 映射有什么优势？

回答： 同一个线程可以处理多个元素，能够复用线程、减少调度与索引开销，并支持处理大于启动线程总数的数据规模，同时保持合并访存。

Lab 2: Matrix Transpose

Exercise 1: 2D Indexing

blockIdx	threadIdx	Global (idx, idy)
(0, 0)	(5, 10)	(5, 10)
(1, 0)	(0, 0)	(32, 0)
(2, 3)	(15, 20)	(79, 116)

题目： 对于 $rows = 100$, $cols = 200$ 的矩阵，需要多少个 blocks？

回答：

$$\left\lceil \frac{200}{32} \right\rceil = 7, \quad \left\lceil \frac{100}{32} \right\rceil = 4, \quad 7 \times 4 = 28 \text{ blocks.}$$

Exercise 2: Index Transformation

Thread (idx, idy)	Input Index	Output Index
(0, 0)	$0 \times 6 + 0 = 0$	$0 \times 4 + 0 = 0$
(1, 0)	$0 \times 6 + 1 = 1$	$1 \times 4 + 0 = 4$
(0, 1)	$1 \times 6 + 0 = 6$	$0 \times 4 + 1 = 1$
(3, 2)	$2 \times 6 + 3 = 15$	$3 \times 4 + 2 = 14$

题目： 为什么写入索引是 $idx * rows + idy$ ，而不是 $idx * cols + idy$ ？

回答： 转置后的输出矩阵尺寸为 $cols \times rows$ ，每一行包含 $rows$ 个元素，因此输出矩阵的行跨度是 $rows$ 。

Exercise 3: Performance Analysis

Test	Dimensions	Elements	Mean Time
1	16×16	256	0.0072 ms
2	128×32	4096	0.0102 ms
3	1×1024	1024	0.0082 ms
4	1001×2001	2003001	0.2370 ms

题目： 大测试的元素数量约为 Test 2 的 500 倍，时间是否也长 500 倍？为什么？

回答： 不是。执行时间约为 $0.2370/0.0102 \approx 23.2$ 倍。小规模测试主要受固定的 kernel 启动开销影响，而大规模测试能够利用 GPU 并行性，所以时间不会随元素数量等比例增长。

Exercise 4: Coalescing Analysis

题目 1： 读取 $input[idy * cols + idx]$ 时，相邻线程是否读取连续地址？

回答： 是。相邻线程的 `idx` 相差 1，因此读取相邻的 `float`，属于合并访存。

题目 2： 写入 `output[idx * rows + idy]` 时，相邻线程写入地址的 `stride` 是多少？

回答： `Stride` 为 1024 个 `float`，即 $1024 \times 4 = 4096$ bytes。

题目 3： 每次内存事务读取 128 bytes 时，一个 `wave32` 的非合并写入需要多少次事务？

回答： 32 次内存事务。

Exercise 5: Tiled Transpose Analysis

题目 1： 为什么共享内存声明为 `tile[BLOCK_SIZE][BLOCK_SIZE + 1]`？

回答： 额外的一列将行跨度从 32 改为 33，使转置访问落到不同的 LDS bank，避免 bank conflict。

题目 2： 写入阶段为什么交换 `blockIdx.x` 和 `blockIdx.y`？

回答： 矩阵转置会交换行和列，因此输入 `tile` 与输出 `tile` 的 `block` 索引对应关系为

$$(\text{blockIdx.x}, \text{blockIdx.y}) \rightarrow (\text{blockIdx.y}, \text{blockIdx.x}).$$

题目 3： `BLOCK_SIZE=32` 时，每个 `block` 使用多少 `shared memory`？

回答：

$$32 \times (32 + 1) \times 4 = 4224 \text{ bytes}.$$

Challenge 1: Tiled Transpose

核心 kernel:

```
__global__ void transpose_tiled(const float* input, float* output,
                               int rows, int cols) {
    __shared__ float tile[BLOCK_SIZE][BLOCK_SIZE + 1];
    int x = blockIdx.x * BLOCK_SIZE + threadIdx.x;
    int y = blockIdx.y * BLOCK_SIZE + threadIdx.y;
    if (x < cols && y < rows)
        tile[threadIdx.y][threadIdx.x] = input[y * cols + x];
    __syncthreads();
    x = blockIdx.y * BLOCK_SIZE + threadIdx.x;
    y = blockIdx.x * BLOCK_SIZE + threadIdx.y;
    if (x < rows && y < cols)
        output[y * rows + x] = tile[threadIdx.x][threadIdx.y];
}
```

Dimensions	Naive	Tiled	Speedup
128×32	0.0101 ms	0.0068 ms	$1.49\times$
1001×2001	0.2364 ms	0.1962 ms	$1.20\times$

Challenge 2: Bank Conflict Analysis

Configuration	Mean Time	Std
With +1 padding	0.1964 ms	0.0009 ms
Without padding	0.2005 ms	0.0067 ms

结论： 去掉 padding 后约慢 2.1%，且波动更大；+1 padding 能减少 LDS bank conflict。

Challenge 3: In-Place Transpose

```
__global__ void transpose_inplace(float* matrix, int n) {
    int x = blockIdx.x * blockDim.x + threadIdx.x;
    int y = blockIdx.y * blockDim.y + threadIdx.y;
    if (x < n && y < n && x < y) {
        float temp = matrix[y * n + x];
        matrix[y * n + x] = matrix[x * n + y];
        matrix[x * n + y] = temp;
    }
}
```

测试结果： In-place transpose: PASSED。

Challenge 4: Rectangular Tile Sizes

采用 32×8 threads，每个线程通过循环处理 tile 中的 4 行：

```
#define TILE_DIM 32
#define BLOCK_ROWS 8
dim3 threadsPerBlock(TILE_DIM, BLOCK_ROWS);
for (int j = 0; j < TILE_DIM; j += BLOCK_ROWS) {
    // load/store row threadIdx.y + j
}
```

Dimensions	32×32	32×8
128×32	0.0068 ms	0.0083 ms
1001×2001	0.1960 ms	0.1929 ms

题目： 矩形 tile 在什么情况下有利？

回答： 矩形 tile 将每个 block 的线程数从 1024 降至 256，可提高调度灵活性，并在大型矩阵或资源压力较大时改善 occupancy；小矩阵中循环开销可能使其更慢。本次大矩阵约快 1.6%。

Challenge 5: Double Buffering

双缓冲使用两个 buffer 和两个 HIP stream，使一个 chunk 的传输与另一个 chunk 的计算重叠：

```
int slot = chunk_number % 2;
hipStreamSynchronize(streams[slot]);
hipMemcpyAsync(d_in[slot], h_in[slot], bytes,
               hipMemcpyHostToDevice, streams[slot]);
transpose_chunk<<<grid, block, 0, streams[slot]>>>(
    d_in[slot], d_out[slot], chunk_rows, cols);
hipMemcpyAsync(h_out[slot], d_out[slot], bytes,
               hipMemcpyDeviceToHost, streams[slot]);
```

Configuration	End-to-End Time
Sequential	18.8894 ms
Double-buffered	15.2274 ms
Speedup	1.24049×
Correctness	PASSED

结论： 双缓冲将时间减少约 19.4%，说明异步传输与 kernel 执行实现了有效重叠。

Lab 3: Histogram

Exercise 1: Analyze the Problem

题目 1: Naive GPU 实现中, 每个输入元素会从 global memory 读取多少次?

回答: `num_bins` 次。

题目 2: `num_bins=256`、 $N = 10\text{M}$ 时, 总 global memory 流量是多少?

回答:

$$10,000,000 \times 256 = 2.56 \times 10^9 \text{ reads}, \quad 2.56 \times 10^9 \times 4 = 10.24 \text{ GB}.$$

题目 3: 为什么这种方法效率低?

回答: 每个 block 都重复扫描整个输入数组, 复杂度为 $O(N \times \text{num_bins})$, 产生大量冗余 global memory 访问。

Exercise 2: Understand the Optimization

题目 1: 为什么 shared-memory `atomicAdd` 比 global-memory `atomicAdd` 更快?

回答: Shared memory 位于片上, 延迟更低、带宽更高, 并减少了 global memory contention。

题目 2: 合并阶段每个 bin 会发生多少次 global `atomicAdd`?

回答: 每个 block 对每个 bin 执行一次, 因此整个 grid 中每个 bin 共执行 `gridDim.x` 次。

题目 3: Grid-stride loop 的作用是什么?

回答: 使有限数量的线程能够覆盖任意大的输入, 并将工作均匀分配给所有线程。

Exercise 3: Record Results

Implementation	Testcase 2	Testcase 4
Serial CPU	2.4549 ms	22.0940 ms
Naive GPU	20.2002 ms	2501.4640 ms
Optimized GPU	0.4683 ms	4.4502 ms
Speedup	Testcase 2	Testcase 4
Optimized vs Serial	5.24×	4.97×
Optimized vs Naive	43.14×	562.10×

Exercise 4: Memory Efficiency

题目： Naive 与 Optimized 的 memory traffic reduction ratio 是多少？

回答：

$$\frac{N \times \text{num_bins}}{N} = \text{num_bins}.$$

例如 256 bins 时，global memory 读取量减少 256×。

Exercise 5: Contention Scenarios

Scenario 1： 所有输入值相同会发生什么？为什么？

回答： 性能下降。所有线程都更新同一个 shared-memory bin，产生最大原子竞争。实测 All-same 为 0.6880 ms，比 Uniform 慢约 46.8%。

Scenario 2： 输入均匀分布时，预期 contention 程度如何？为什么？

回答： Contention 较低，因为原子更新分散在不同 bin 上。

Challenge 1: Warp-Level Histogram

核心 kernel:

```
__global__ void kernel_warp(const int* input, int* histogram,
                           int N, int num_bins) {
    extern __shared__ int warp_hist[];
    int tid = blockIdx.x * blockDim.x + threadIdx.x;
    int stride = blockDim.x * gridDim.x;
    int warp_id = threadIdx.x / 32;
    int warps = blockDim.x / 32;

    for (int i = threadIdx.x; i < warps * num_bins;
         i += blockDim.x)
        warp_hist[i] = 0;
    __syncthreads();

    for (int i = tid; i < N; i += stride)
        atomicAdd(&warp_hist[warp_id * num_bins + input[i]], 1);
    __syncthreads();

    for (int bin = threadIdx.x; bin < num_bins;
         bin += blockDim.x) {
        int sum = 0;
        for (int w = 0; w < warps; ++w)
            sum += warp_hist[w * num_bins + bin];
        atomicAdd(&histogram[bin], sum);
    }
}
```

```
}
}
```

Implementation	Testcase 2	Testcase 4
Block-private	0.4707 ms	4.4486 ms
Warp-private	0.4577 ms	4.3888 ms

结论： Warp-private 正确性验证通过，在两个测试中分别快约 2.8% 和 1.3%。

Challenge 2: Different Distributions

Distribution	Mean Time	Std
Uniform	0.4686 ms	0.0035 ms
Gaussian	0.4641 ms	0.0094 ms
Skewed 90%	0.6134 ms	0.0018 ms
All-same	0.6880 ms	0.0047 ms

结论： 数据越集中，多个线程更新同一 bin 的概率越高。Skewed 90% 比 Uniform 慢约 30.9%，All-same 的 contention 最大且最慢。

Challenge 3: Vary Number of Bins

Bins	Effective Shared Storage	Mean Time	Std
16	64 bytes	0.4595 ms	0.0071 ms
64	256 bytes	0.4550 ms	0.0069 ms
256	1024 bytes	0.4508 ms	0.0089 ms
1024	4096 bytes	0.4520 ms	0.0093 ms

题目 1： Bin 数量如何影响 shared memory 使用量？

回答： 有效存储量按 $4 \times \text{num_bins}$ bytes 线性增长。当前 kernel 静态声明 1024 个 int，因此物理上始终预留 4096 bytes，但只使用前 num_bins 项。

题目 2： 优化在何时开始变得不够有效？

回答： 本次 16–1024 bins 的平均时间均约为 0.45 ms，没有观察到明显失效点。若 bin 数继续增加，shared-memory 占用和合并开销会增大并降低 occupancy，此时优化会逐渐失效。

Lab 4: Parallel Reduction

Exercise 1: Understand Tree Reduction

题目 1: 1024 个元素、block size 256 时, 每个 block 内有多少个 reduction steps?

回答: $\log_2 256 = 8$ steps。

题目 2: Block-level reduction 后剩余多少个 partial sums?

回答: $\lceil 1024/256 \rceil = 4$ 个 partial sums。

题目 3: 为什么每个 reduction step 后都要使用 `__syncthreads()`?

回答: 确保当前步骤的所有 shared-memory 写入完成后, 下一步才读取结果, 避免数据竞争。

Exercise 2: Occupancy Analysis

Block Size	Warps/Block	Occupancy	Time
64	2	100%	0.0731 ms
128	4	100%	0.0591 ms
256	8	100%	0.0633 ms
512	16	100%	0.0730 ms
1024	32	100%	0.0771 ms

题目: 更高的 occupancy 是否总能带来更好的性能?

回答: 不是。所有配置的 occupancy 均为 100%, 但 block size 128 最快; 性能还取决于同步次数、活跃线程比例和 atomic 操作数量。

Exercise 3: Trade-offs

题目 1: $N = 1,000,000$ 、block size 64 时有多少次最终 atomic 操作?

回答: $\lceil 1,000,000/64 \rceil = 15625$ 次。

题目 2: Block size 1024 时有多少次最终 atomic 操作?

回答: $\lceil 1,000,000/1024 \rceil = 977$ 次。

题目 3: 为什么更少的 atomic 操作可能提高性能?

回答: 更少的 global atomic 操作可以减少竞争和串行化开销。

Optimization Benchmark

Strategy	Time	Speedup vs Naive
Naive Atomic	0.0846 ms	1.00×
Tree (Shared Memory)	0.0632 ms	1.34×
Tree + Warp Shuffle	0.0545 ms	1.55×
Multi-Element + Tree + Shuffle	0.0466 ms	1.82×

验证： 四种策略结果均为 -74993790 , Verification: ALL PASSED。

Exercise 4: Algorithm Analysis

题目 1： Tree reduction 第一步之后有多少比例的线程空闲？

回答： 50%。

题目 2： 为什么 warp shuffle 不需要 `__syncthreads()`？

回答： 同一 wavefront 内的线程锁步执行，shuffle 指令直接在 lane 的寄存器之间交换数据。

题目 3： 每个线程处理多个元素如何改善内存带宽利用率？

回答： 它减少了 blocks、partial sums 和最终 atomic 操作，摊薄索引与同步开销；grid-stride 访问仍保持相邻线程读取相邻地址。

Challenge 1: Tree Reduction

```
extern __shared__ int sdata[];
sdata[tid] = (idx < N) ? input[idx] : 0;
__syncthreads();
for (int stride = blockDim.x / 2; stride > 0; stride >>= 1) {
    if (tid < stride)
        sdata[tid] += sdata[tid + stride];
    __syncthreads();
}
if (tid == 0)
    atomicAdd(output, sdata[0]);
```

结果： 0.0632 ms, 相比 Naive 加速 1.34×

Challenge 2: Warp Shuffle

```

if (tid < warpSize) {
    int value = sdata[tid];
    for (int offset = warpSize / 2; offset > 0; offset >>= 1)
        value += __shfl_down(value, offset);
    if (tid == 0)
        atomicAdd(output, value);
}

```

结果： 0.0545 ms, 相比 Naive 加速 1.55 $\times$ 。

Challenge 3: Multi-Pass Reduction

第一阶段每个 block 输出一个 partial sum, 第二阶段使用一个 block 对 partial sums 进行最终归约。

N	Partial Sums	Kernel Time	Verification
1M	1954	0.0445 ms	PASSED
100M	195313	4.6521 ms	PASSED

结论： 在 $N = 1\text{M}$ 时, Multi-Pass 比单 kernel 的 Multi-Element 0.0466 ms 略快。100M 输入的数学和超出 32 位 int 范围, 输出为溢出后的 -1984960864 , 但 GPU 与预期的 32 位结果一致。

Challenge 4: Different Operations

Operation	Time	Result	Status
Sum	0.0459 ms	333332	PASSED
Max	0.0458 ms	1	PASSED
Min	0.0459 ms	-1	PASSED
Product	0.0462 ms	1	PASSED

结论： 四种操作的执行时间非常接近。Product 测试仅使用 1 和 -1, 以避免整数溢出。

Lab 5: Monte Carlo Integration

Exercise 1: Understand the Algorithm

题目 1: 为什么每个线程都除以 `n_samples`，而不是最后只除一次？

回答: 这样每个线程可以直接产生对最终积分的贡献并立即执行 `atomicAdd`，无需最后再缩放。数学上可以先求和再只除一次，后者通常更高效且舍入误差更小。

题目 2: Atomic reduction 的时间复杂度是多少？

回答: 总工作量为 $O(N)$ ，但所有线程竞争同一地址，atomic 阶段高度串行化。

题目 3: 如何使用 shared-memory reduction 替代每线程 atomic？

回答: 每个 block 先在 shared memory 中进行树形归约，最后仅由每个 block 的 thread 0 对 global result 执行一次 `atomicAdd`。

Exercise 2: Manual Calculation

Test 1 的样本和为

$$4.1805 + 1.7864 + 4.7554 + 3.9983 + 2.2786 + 1.1873 + 4.1222 + 1.442 = 23.7507.$$

预期结果为

$$\frac{2}{8} \times 23.7507 = 5.937675.$$

回答: 与程序输出 5.937675 完全一致。Test 2 中 $a = b = 1$ ，区间宽度为 0，因此结果为 0。

Exercise 3: Scalability Analysis

Test	Sample Count	Naive Kernel Time
4	100000	0.4124 ms
7	10000000	39.9198 ms
8	100000000	398.3132 ms

题目 1: 样本数从 Test 4 到 Test 7 增加 100 倍，执行时间增加多少？

回答: $39.9198/0.4124 \approx 96.8$ 倍。

题目 2: 扩展是否呈线性？为什么？

回答: Naive atomic kernel 基本呈线性增长，因为每个样本都要对同一地址执行一次 `atomicAdd`，atomic contention 成为主要瓶颈。

Exercise 4: Bottleneck Analysis

题目 1: 计算耗时 1 cycle、atomic 耗时 100 cycles 时, 理论效率是多少?

回答:

$$\frac{1}{1+100} \times 100\% \approx 0.99\%.$$

题目 2: Shared-memory reduction 如何改善性能?

回答: 先在 block 内进行 shared-memory 归约, 将 global atomic 从每线程一次降低为每 block 一次。

题目 3: $N = 1\text{M}$ 、256 threads/block 时需要多少次 global atomics?

回答: $\lceil 1,000,000/256 \rceil = 3907$ 次。

Exercise 5: Precision Considerations

题目 1: Double precision 提供多少位有效数字?

回答: 约 15–16 位十进制有效数字。

题目 2: 改用 float 处理 100M 样本可能出现什么问题?

回答: Float 只有约 6–7 位有效数字, 大量累加可能造成明显舍入误差, 小贡献可能丢失; 并行 atomic 的非固定累加顺序还会导致结果出现小幅波动。

Exercise 6: Optimization Analysis

题目 1: $N = 1\text{M}$ 、256 blocks 时有多少次 global atomics?

回答: 256 次。

题目 2: Grid-stride loop 的作用是什么?

回答: 让有限数量的线程覆盖任意数量的样本, 并将工作均匀分配给线程。

题目 3: 为什么缩放因子只由 thread 0 乘一次?

回答: 每个 block 只对归约后的 partial sum 缩放一次, 减少重复除法与舍入误差。

Challenge 1: Shared-Memory Reduction

Samples	Naive	Shared	Speedup
100000	0.4124 ms	0.0426 ms	9.68×
10000000	39.9198 ms	0.9093 ms	43.90×
100000000	398.3132 ms	8.3861 ms	47.50×

结论： 样本量越大，Naive atomic contention 越严重，shared-memory reduction 的优势越明显。

Challenge 2: Compute Pi

Item	Result
Samples	10000000
Points inside circle	7854128
Estimated π	3.1416512000
Absolute error	0.0000585464
Kernel time	1.9550 ms

Challenge 3: Error Analysis

对 $\int_0^1 x^2 dx = 1/3$ 的 Monte Carlo 估计结果：

N	Estimate	Absolute Error	RMSE	RMSE $\sqrt{N}$
1000	0.3203345839	0.0129987495	0.0110238523	0.3486048175
10000	0.3279692420	0.0053640914	0.0031565892	0.3156589227
100000	0.3318039658	0.0015293675	0.0010015858	0.3167292392
1000000	0.3326602739	0.0006730594	0.0004252395	0.4252395382

结论： 绝对误差随 N 增大总体下降，RMSE $\sqrt{N}$ 大致稳定，符合 $\text{Error} \propto 1/\sqrt{N}$ 的收敛规律。

Challenge 4: Float vs Double

在 $N = 100000000$ 时：

Type	Estimate	Absolute Error	Time
Double	0.3333297620	0.0000035713	5.1762 ms
Float	0.3333297372	0.0000035961	0.7372 ms

结论： Float 加速 7.02×，误差略高；本次误差仍主要来自 Monte Carlo 采样，而不是浮点格式差异。

Lab 6: K-Means Clustering

Exercise 1: Algorithm Analysis

题目 1: Assignment kernel 中每线程的时间复杂度是多少？当 $k = 100$ 时执行多少次距离计算？

回答: 复杂度为 $O(k)$ ；当 $k = 100$ 时，每线程执行 100 次距离计算。

题目 2: 为什么 centroid 更新先使用 shared-memory atomics？

回答: 先在每个 block 内合并结果，可将 global atomic 从每点 3 次降低为每 block、每 centroid 3 次。

题目 3: 如果某个 centroid 没有被分配任何点，会发生什么？

回答: 该 centroid 保留上一轮的位置。

Exercise 2: Performance Analysis

Item	Value
Sample size	50000
Clusters	50
Max iterations	100
Execution time	0.102 s

题目: 每轮迭代中占主导的计算是什么？

回答: Assignment kernel 占主导。每个点都要与全部 k 个 centroid 计算距离，复杂度为 $O(Nk)$ ；本测试每轮包含 2500000 次距离计算。

Exercise 3: Compute Intensity

Item	Result
Distance calculations per point	50
Total distance calculations	2500000
Shared-memory atomics	150000
Blocks, $\lceil 50000/256 \rceil$	196
Global atomics, $196 \times 50 \times 3$	29400

Exercise 4: Memory Optimization

题目 1: 每个线程都会读取 centroids，如何优化访问模式？

回答： 将 centroids 缓存到每个 block 的 shared memory，或使用只读/constant cache，避免每线程重复从 global memory 加载。

题目 2： $k = 1000$ 时对 cache 性能有何影响？

回答： 每线程要扫描大量 centroid，cache 压力和 cache miss 增加，global memory 流量及计算量都会显著上升。

Exercise 5: Convergence Analysis

题目 1： 当前实现的收敛阈值是多少？

回答： $dx^2 + dy^2 < 10^{-8}$ ，即 centroid 移动距离小于 10^{-4} 。

题目 2： 为什么需要检测收敛，而不是总运行 `max_iterations`？

回答： 提前停止可以避免无效迭代，减少距离计算、atomic 操作和 kernel 启动。

题目 3： 交换 prev/new centroid buffers 有什么作用？

回答： 通过交换指针复用两组 buffer，避免每轮复制 centroid 数据。

Challenge 1: Support Larger k

动态 shared memory 布局：

```
extern __shared__ unsigned char shared_raw[];
float* shared_x = reinterpret_cast<float*>(shared_raw);
float* shared_y = shared_x + k;
int* shared_count = reinterpret_cast<int*>(shared_y + k);
size_t shared_bytes = k * (2 * sizeof(float) + sizeof(int));
```

Item	Result
k	1000
Dynamic shared memory	12000 bytes
Kernel time	0.1571 ms
Verification	PASSED

Challenge 2: Inertia Output

$$\text{inertia} = \sum_i \|x_i - \mu_{\text{label}_i}\|^2.$$

Test	Inertia	End-to-End Time
Test 1	1.87278×10^7	–
Test 4	6.58608×10^8	0.097 s

Challenge 3: Extend to 3D

三维距离计算为

```
float dx = x[i] - cx[c];
float dy = y[i] - cy[c];
float dz = z[i] - cz[c];
float distance = dx * dx + dy * dy + dz * dz;
```

Item	Result
Centroid 0	(1, 1, 1)
Centroid 1	(11, 11, 11)
Labels	0, 0, 0, 1, 1, 1
Verification	PASSED

Challenge 4: K-Means++ Initialization

GPU 并行维护每个点到已选 centroid 的最小距离平方, 并按该距离平方作为权重选择后续 centroid。

Centroid	Selected Index	Position
0	4	(10, 10)
1	10	(-10, 11)
2	3	(1, 1)

验证: 三个 centroid 分别来自三个不同数据簇, PASSED。

Challenge 5: Mini-Batch K-Means

Method	Training Time	Inertia
Full batch	515.7490 ms	166998.3281
Mini batch	70.0453 ms	167008.7812

结论: Mini-batch 加速 7.36 $\times$, inertia ratio 为 1.000063, 即 inertia 仅增加约 0.0063%, 显著缩短训练时间且聚类质量几乎不变。